COMP-SCI-5565
Intro to Statistical Learning

Topic 9 – Deep Learning

Adu Baffour, PhD

University of Missouri-Kansas City
Division of Computing, Analytics, and Mathematics
School of Science and Engineering
aabnxq@umkc.edu

UMKC

Lecture Objectives

- Describe the structure of a single-layer neural network.
- Describe the structure of a multilayer neural network.
- Describe the structure of a convolutional neural network.
- Describe the structure of a recurrent neural network.
- Describe the structure of a transformer network.
- Recognize the process by which neutral networks are fit.
- Explain the double descent phenomenon.

Outline

9.1 Introduction

9.2 Single Layer Neural Network

9.3 Multilayer Neural Network

9.4 Convolutional Neural Network

9.5 Recurrent Neural Network

9.6 Transformer Network

9.7 Tricks of the Trade

Outline

9.1 Introduction

9.2 Single Layer Neural Network

9.3 Multilayer Neural Network

9.4 Convolutional Neural Network

9.5 Recurrent Neural Network

9.6 Transformer Network

9.7 Tricks of the Trade

9.1 Introduction

- Neural networks became popular in the 1980s.
- Then came SVMs, Random Forests, and Boosting in the 1990s, and Neural Networks took a back seat.
- Re-emerged around 2010 as Deep Learning. By the 2020s, it was very dominant and successful.
- Part of the success is due to vast improvements in computing power, larger training sets, and software: Tensorflow and PyTorch.

Much of the credit goes to three pioneers and their students: Yann LeCun, Geoffrey Hinton, and Yoshua Bengio, who received the 2019 ACM Turing Award for their work in Neural Networks.

UMKC

Outline

9.1 Introduction

9.2 Single Layer Neural Network

9.3 Multilayer Neural Network

9.4 Convolutional Neural Network

9.5 Recurrent Neural Network

9.6 Transformer Network

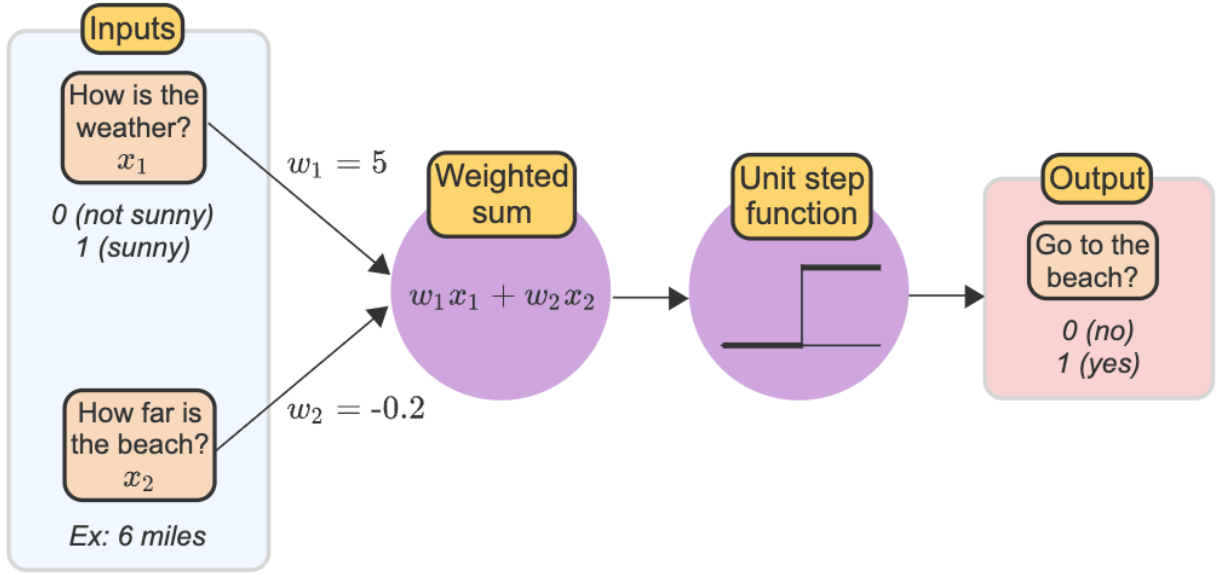
9.7 Tricks of the Trade

9.2 Single Layer Neural Network

- A single-layer neural network contains one layer or set of artificial neurons.

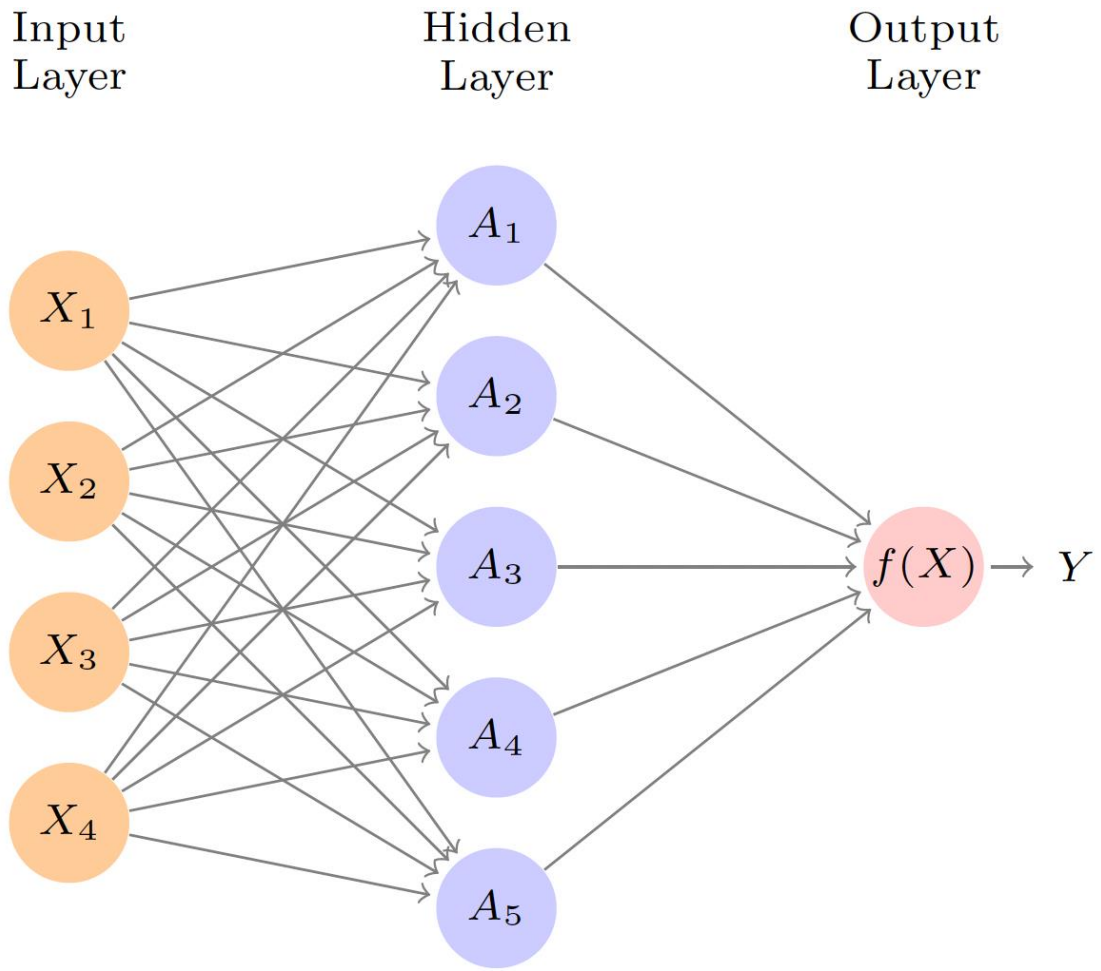
What is an artificial neuron?

- Like a biological neuron, an artificial neuron receives inputs and produces an output.
- The weighted sum is obtained from the inputs, and a threshold function or activation, such as the unit step function, compares the weighted sum to a threshold.



9.2 Single Layer Neural Network

- Let's consider a dataset made of p predictors $X = (X_1, X_1, \ldots, X_p)$
- Each neuron A_k in a hidden layer computes $A_k = g\left(\sum_{j=1}^{p} X_j W_{kj} + b_{k0}\right)$, where $g(z)$ is a non-linear function.
- To obtain an output layer, we use activations A_k as inputs, resulting in a function $\mathrm{f(X)} = \sum_{k=1}^{K} \beta_k A_k + \beta_0$.
- W_{kj} and β_k need to be estimated from data.



9.2 Single Layer Neural Network

Activation function

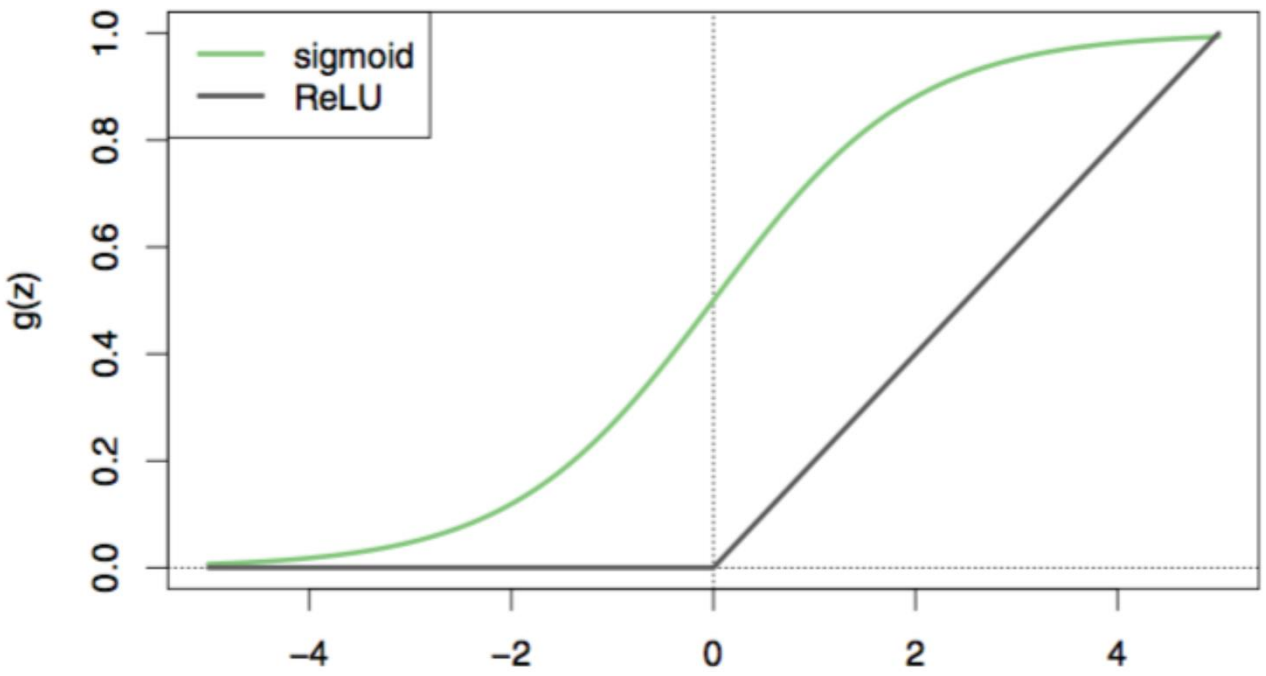
- To introduce non-linearity into the model.
- There are various options, but the most used ones are:

(1) Sigmoid

$$g(z) = \frac{e^z}{1+e^z}$$

(2) ReLU rectified linear unit

$$g(z) = (z)_+ = \begin{cases} 0 & \text{if } z < 0 \\ z & \text{otherwise} \end{cases}$$

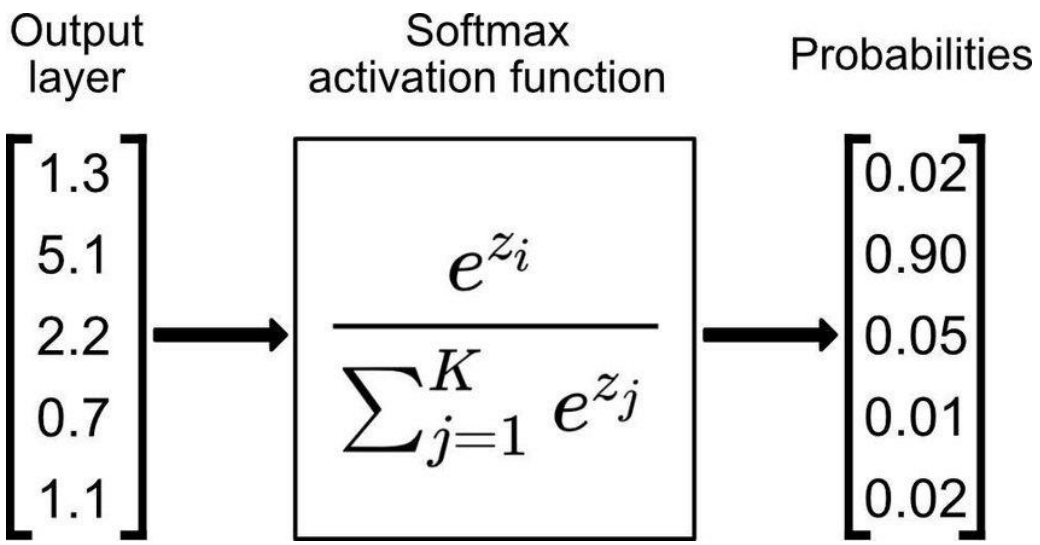


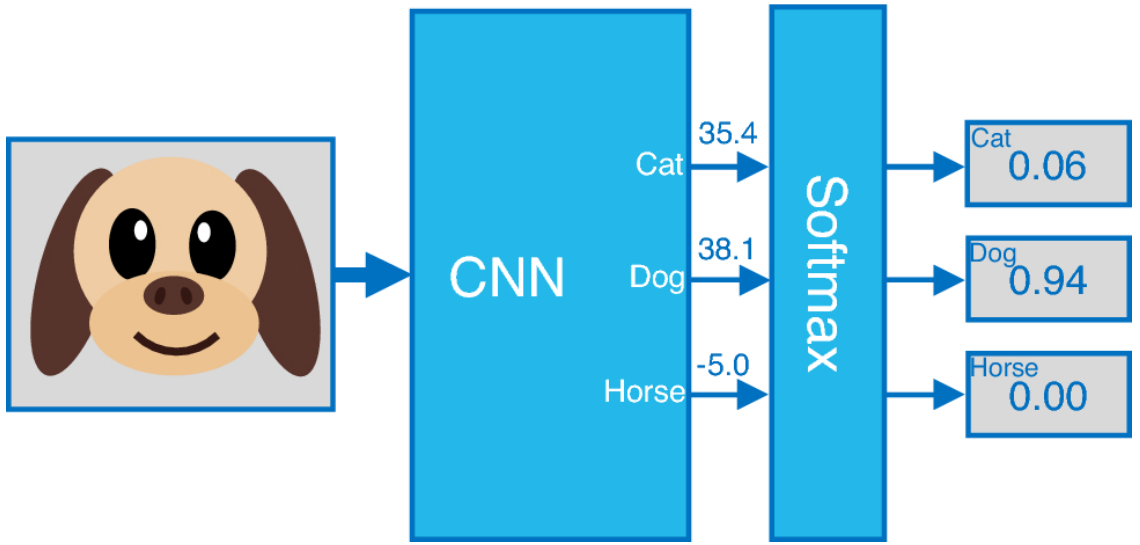
UMKC

9.2 Single Layer Neural Network

Activation function (Cont.)

(3) Softmax: turns a vector of K real values into a vector of K real values that sum to 1. It is usually used in the output layer.





UMKC

Outline

9.1 Introduction

9.2 Single Layer Neural Network

9.3 Multilayer Neural Network

9.4 Convolutional Neural Network

9.5 Recurrent Neural Network

9.6 Transformer Network

9.7 Tricks of the Trade

9.3 Multilayer Neural Network

First hidden layer for

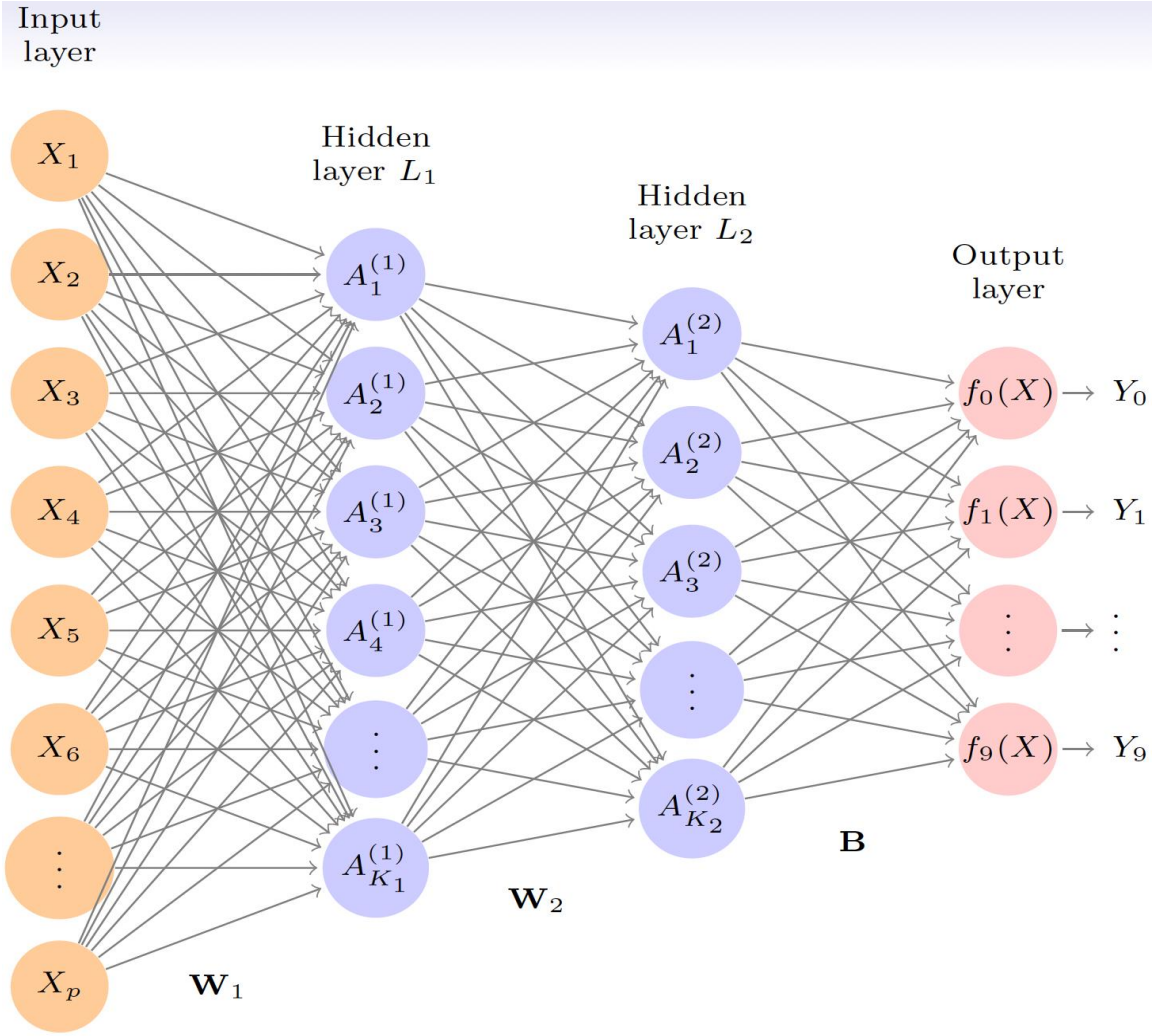
$k = 1, \ldots, K_1$

$$A_k^1 = h_k^1(X)$$

Second hidden layer for

$k = 1, \ldots, K_2$

$$A_k^2 = h_k^2(X)$$



Outline

9.1 Introduction

9.2 Single Layer Neural Network

9.3 Multilayer Neural Network

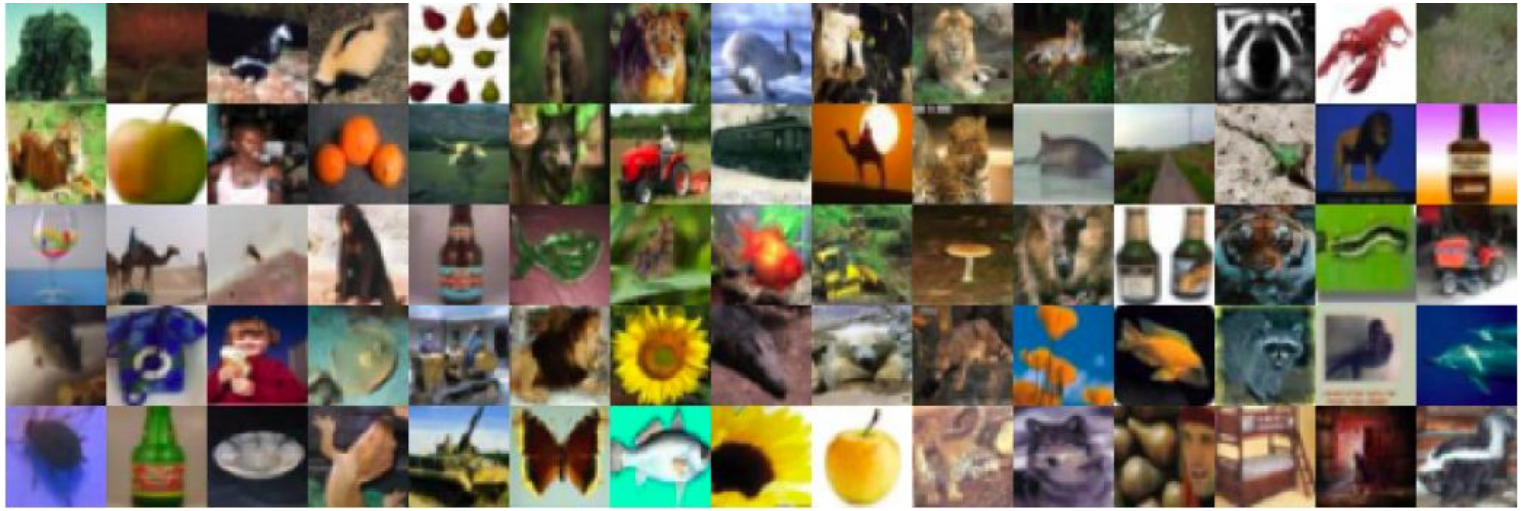
9.4 Convolutional Neural Network

9.5 Recurrent Neural Network

9.6 Transformer Network

9.7 Tricks of the Trade

9.4 Convolutional Neural Network



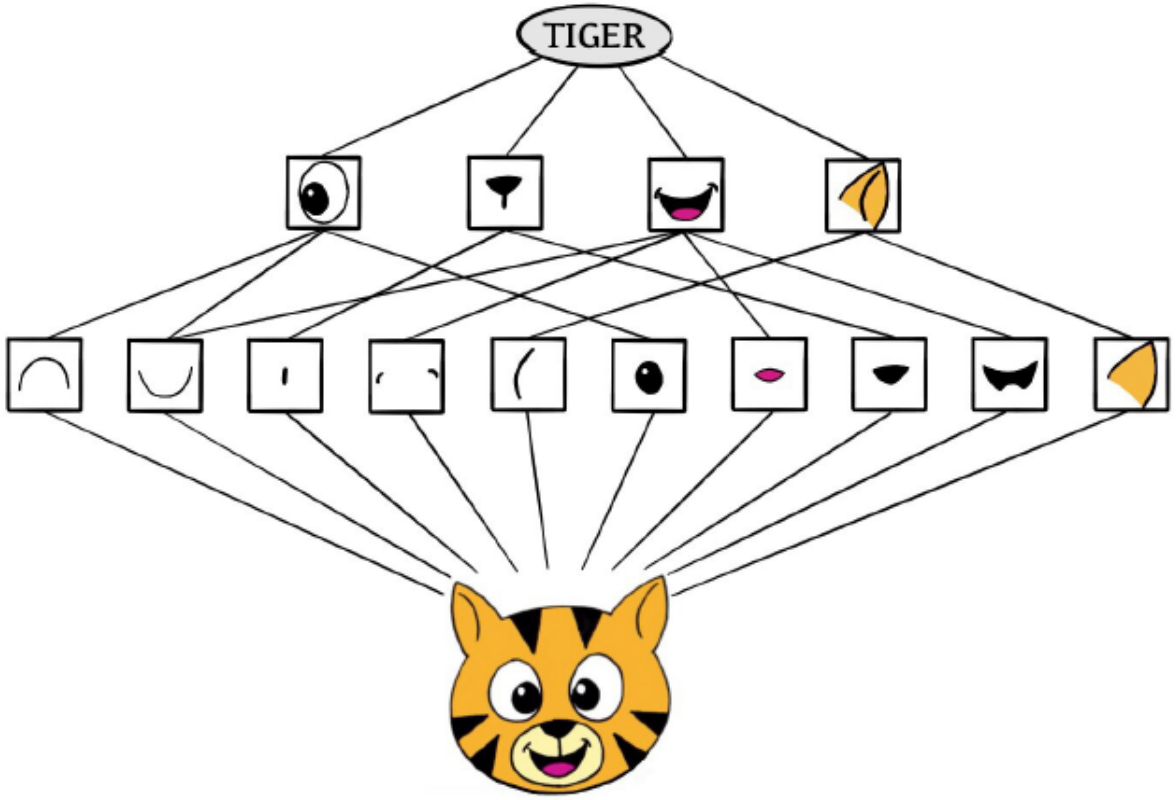
- A major success story for classifying images.
- Shown are samples from CIFAR100 database. 32×32 color natural images, with 100 classes.
- 50K training images and 10K test images.
- Each image is a three-dimensional array or feature map: $32 \times 32 \times 3$ array of 8-bit numbers.
- The last dimension represents the three color channels red, green , and blue.

UMKC

9.4 Convolutional Neural Network

How CNNs work

- The CNN builds up an image in a hierarchical fashion.
- The network first identifies low-level features like edges and shapes in the input image.
- These features are then combined to form higher-level features.
- This hierarchical construction is achieved using convolution and pooling layers.



UMKC

9.4 Convolutional Neural Network

Convolution

- Convolution operation uses filters/kernels.
- Convolutional filters determine whether a local feature is present in the image.
- We can have:
  - K different convolution filters in the first hidden layer
  - typically apply the ReLU activation function

9.4 Convolutional Neural Network

Convolution (cont.)

$$\text{Input Image} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \\ j & k & l \end{bmatrix} \quad \text{Convolution Filter} = \begin{bmatrix} \alpha & \beta \\ \gamma & \delta \end{bmatrix}.$$

$$\text{Convolved Image} = \begin{bmatrix} a\alpha + b\beta + d\gamma + e\delta & b\alpha + c\beta + e\gamma + f\delta \\ d\alpha + e\beta + g\gamma + h\delta & e\alpha + f\beta + h\gamma + i\delta \\ g\alpha + h\beta + j\gamma + k\delta & h\alpha + i\beta + k\gamma + l\delta \end{bmatrix}$$

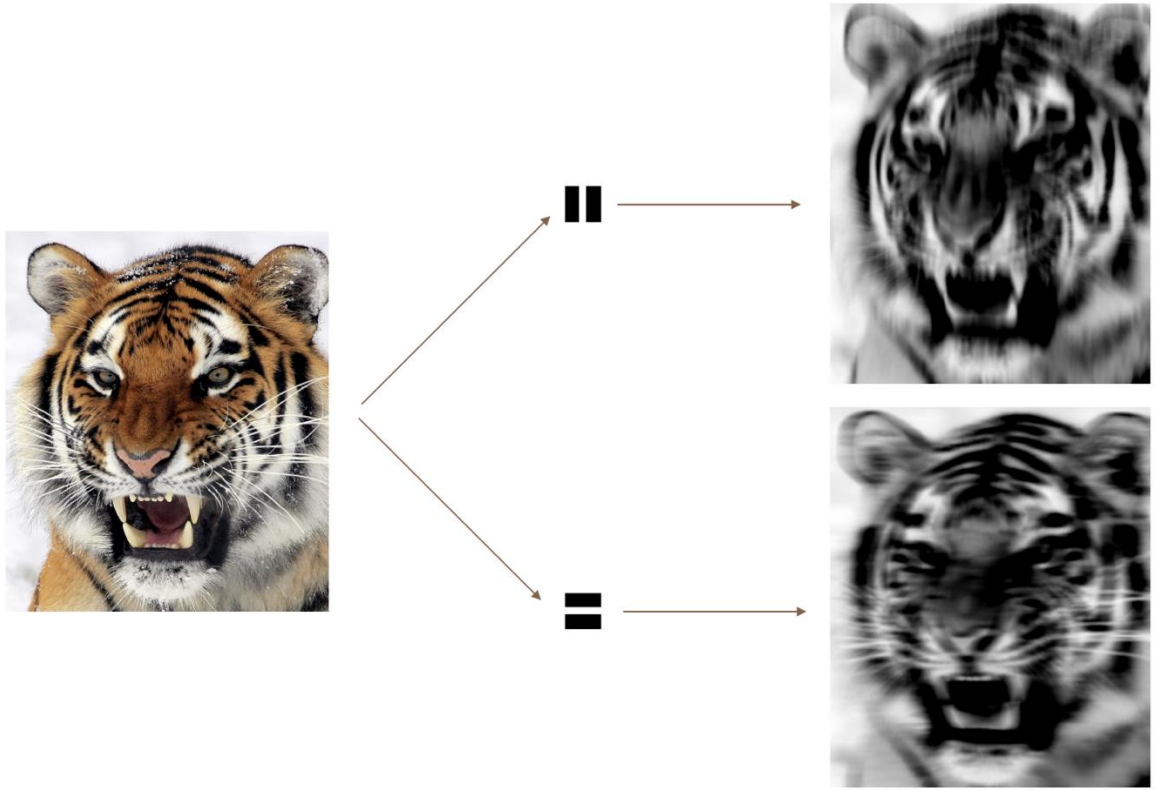
- The filter is an image representing a small shape, edge, etc.
- We slide it around the input image, scoring for matches.
- The scoring is done via *dot-products*, illustrated above.
- If the sub-image of the input image is like the filter, the score is high; otherwise, it is low.
- The filters are *learned* during training.

UMKC

9.4 Convolutional Neural Network

Convolution (cont.)

- The idea of convolution with a filter is to find common patterns that occur in different parts of the image.
- The two filters shown here highlight vertical and horizontal stripes.
- The result of the convolution is a new feature map.
- Since images have three color channels, the later also does one filter per channel, and dot-products are summed.
- The network learns the weights in the filters.



9.4 Convolutional Neural Network

Pooling

- Pooling Layers reduce the size of the image by a factor of two in each direction and provide some location invariance.

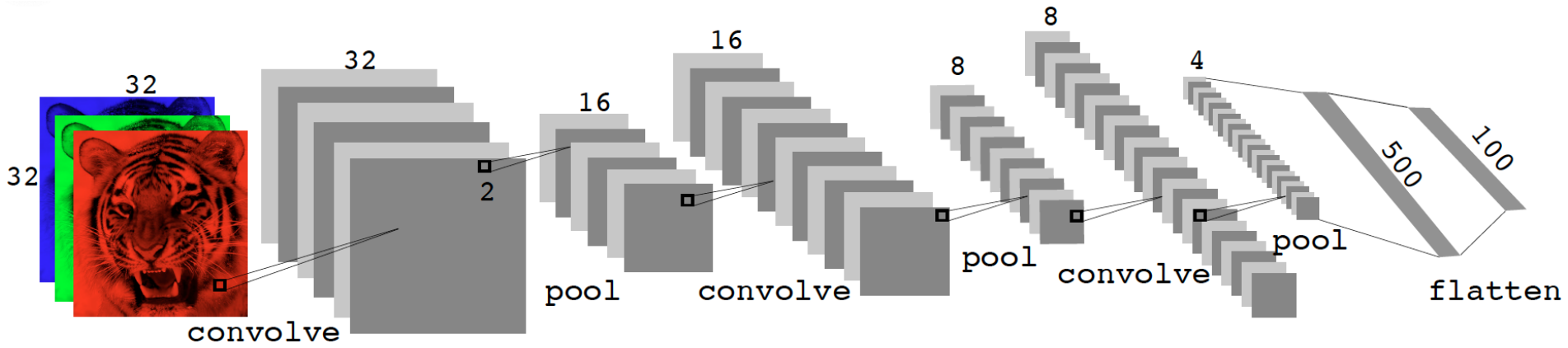
$$\text{Max pool}\begin{bmatrix}1 & 2 & 5 & 3\\ 3 & 0 & 1 & 2\\ 2 & 1 & 3 & 4\\ 1 & 1 & 2 & 0\end{bmatrix} \rightarrow \begin{bmatrix}3 & 5\\ 2 & 4\end{bmatrix} \qquad \text{Avg pool}\begin{bmatrix}1 & 2 & 5 & 3\\ 3 & 0 & 1 & 2\\ 2 & 1 & 3 & 4\\ 1 & 1 & 2 & 0\end{bmatrix} \rightarrow \begin{bmatrix}1.5 & 2.75\\ 1.25 & 2.25\end{bmatrix}$$

- Each non-overlapping 2 2 block is replaced by its maximum.
- This sharpens the feature identification.
- Allows for locational invariance.
- Reduces the dimension by a factor of 4 – i.e., factor of 2 in each dimension.

UMKC

9.4 Convolutional Neural Network

Architecture of a CNN



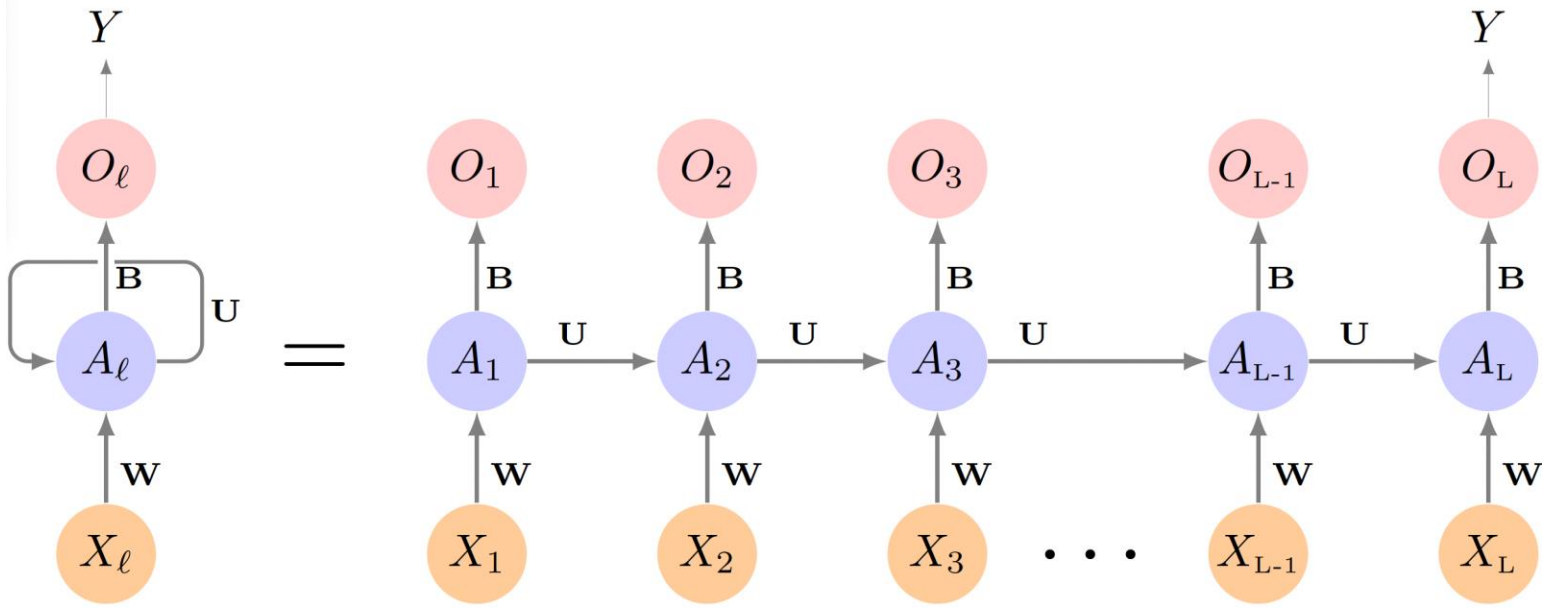
- Many convolve + pool layers.
- Filters are typically small, e.g., each channel 3×3.
- Each filter creates a new channel in the convolution layer.
- As pooling reduces size, the number of filters/channels is typically increased.
- The number of layers can be very large. E.g., resnet152 trained on imagenet 1000-class image data base has 152 layers!

UMKC

Outline

9.1 Introduction
9.2 Single Layer Neural Network
9.3 Multilayer Neural Network
9.4 Convolutional Neural Network
9.5 Recurrent Neural Network
9.6 Transformer Network
9.7 Tricks of the Trade

9.5 Recurrent Neural Network



- $X = \{X_1, X_2, \dots, X_L\}$ is a sequence.
- Hidden layer is a sequence $\{A_l\}_1^L = \{A_1, A_2, \dots, A_L\}$.
- Each A_l feeds into the output layer and produces a prediction for O_l for Y
- Loss function: $(Y - O_L)^2$

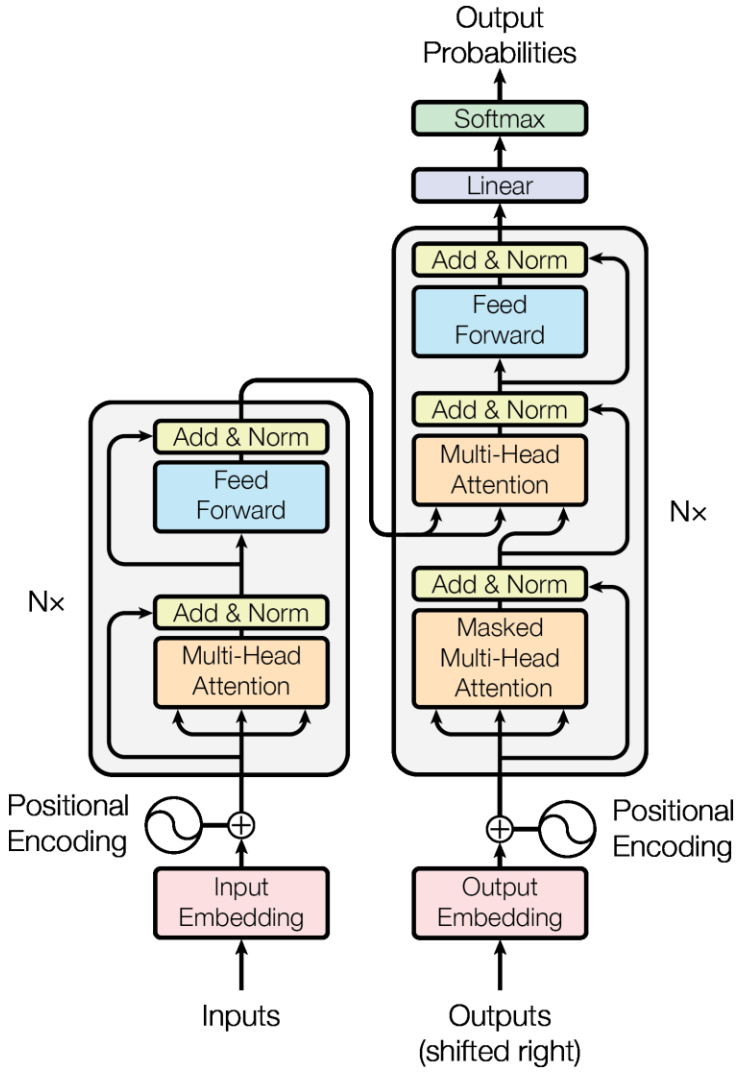
UMKC

Outline

9.1 Introduction
9.2 Single Layer Neural Network
9.3 Multilayer Neural Network
9.4 Convolutional Neural Network
9.5 Recurrent Neural Network
9.6 Transformer Network
9.7 Tricks of the Trade

9.6 Transformer Network

- The Transformer was proposed in the paper Attention is All You Need
- Great Blog post - https://jalammar.github.io/illustrated-transformer/
- Explained visually - https://www.youtube.com/watch?v=wjZofJX0v4M
- Transformer Explainer - https://poloclub.github.io/transformer-explainer/
- PyTorch Implementation - https://nlp.seas.harvard.edu/2018/04/03/attention.html



Outline

9.1 Introduction

9.2 Single Layer Neural Network

9.3 Multilayer Neural Network

9.4 Convolutional Neural Network

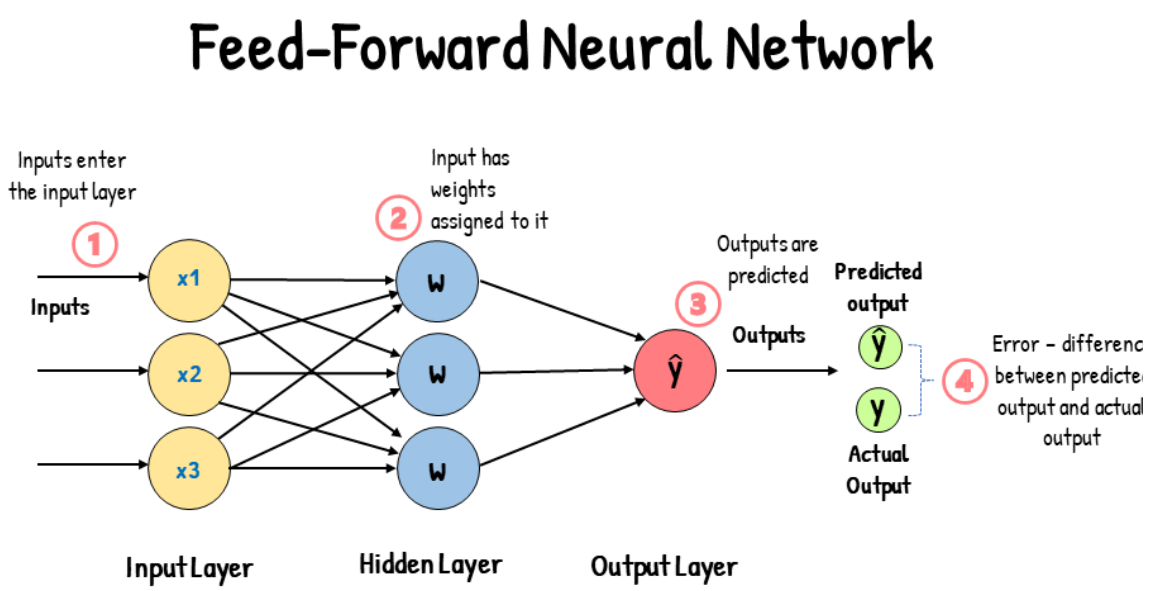
9.5 Recurrent Neural Network

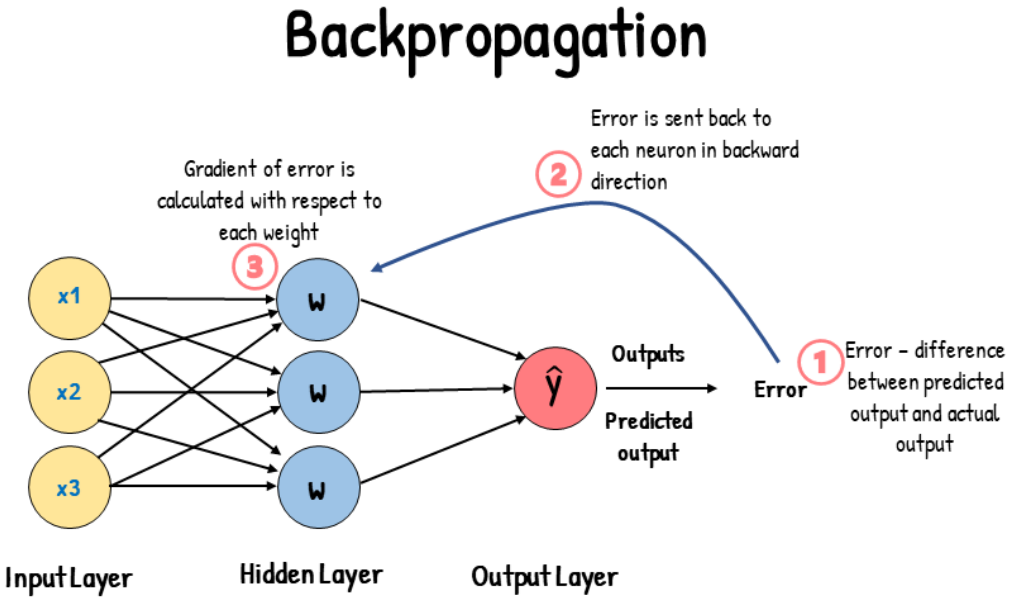
9.6 Transformer Network

9.7 Tricks of the Trade

9.7 Tricks of the Trade

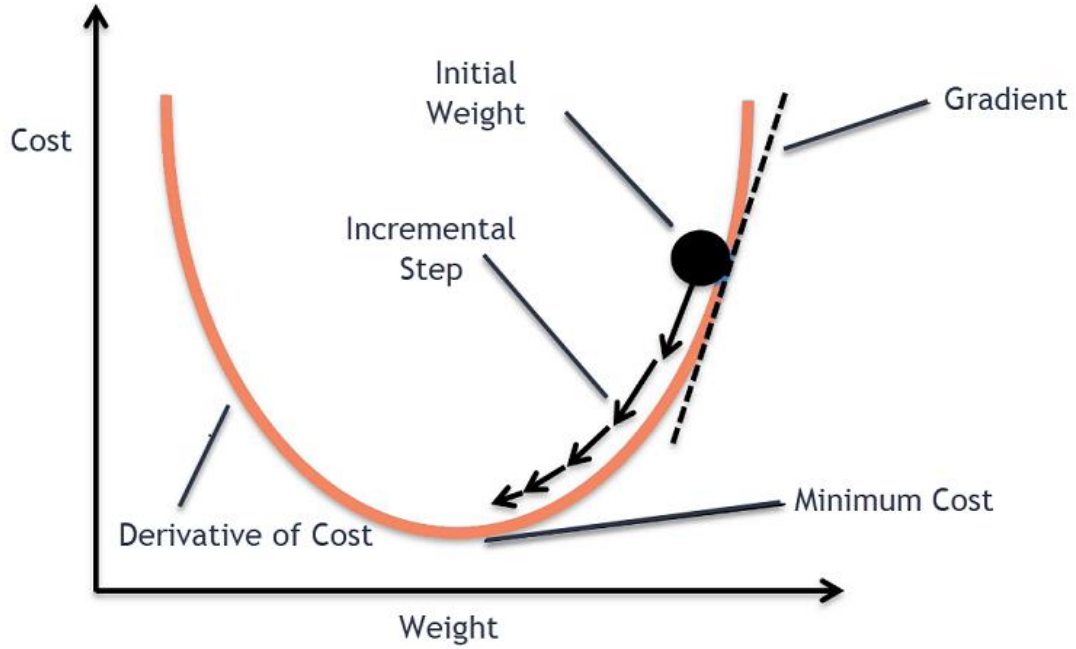
- Backpropagation. Backpropagation is an iterative algorithm used to train artificial neural networks to minimize the cost function by determining which weights and biases should be adjusted.
- During every epoch, it takes the error rate of forward propagation and feeds this loss backward through the neural network layers to fine-tune the weights by moving down toward the gradient of the error.





9.7 Tricks of the Trade

- Gradient descent is one of the backpropagation algorithms used to train deep learning models by finding a local minimum of a differentiable function.



- Slow learning. Gradient descent is slow, and a small learning rate slows it further. With early stopping, this is a form of regularization.

UMKC

9.7 Tricks of the Trade

- Stochastic gradient descent. Rather than compute the gradient using all the data, use a small minibatch drawn randomly at each step.
- An epoch is a count of iterations and amounts to the number of minibatch updates such that n samples in total have been processed, i.e., 60K=128 469 for MNIST.
- Regularization. Ridge and lasso regularization can shrink the weights at each layer. Two other popular forms of regularization are dropout and augmentation.

9.7 Tricks of the Trade

Software

- Wonderful software is available for neural networks and deep learning.
- Tensorflow from Google and PyTorch from Facebook are the two popular ones.
- Both are Python packages.
- There are many other packages.

Further Reading

- Chapter 10 – James G., Witten D., Hastie T., Tibshirani R., & Taylor J. (2023) *An Introduction to Statistical Learning with Applications in Python.* Springer https://link.springer.com/book/10.1007/978-3-031-38747-0
- Chapter 10 - Introduction to Statistical Learning Using R Book Club: https://r4ds.github.io/bookclub-islr/support-vector-machines.html
- Neural Networks - https://www.simplilearn.com/tutorials/deep-learning-tutorial/neural-network
- Backpropagation: https://www.youtube.com/watch?v=Ilg3gGewQ5U
- Gradient Descent: https://builtin.com/data-science/gradient-descent
- CNN - https://www.simplilearn.com/tutorials/deep-learning-tutorial/convolutional-neural-network
- RNN - https://www.simplilearn.com/tutorials/deep-learning-tutorial/rnn
- Segment Anything - https://segment-anything.com/

UMKC

End of Topic

Thank you

Any questions?

UMKC